

Discrete Mathematics and Graph Theory

Module 4 (CO4) — Trees and Networks

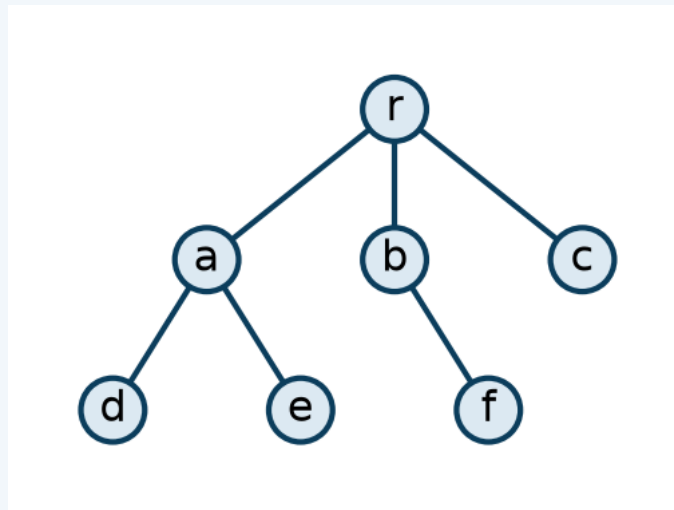
Questions with Detailed Step-by-Step Solutions

Every question is solved from zero to the final answer — what is used, why it is used, and every small step explained. Part A questions carry 2 marks; Part B questions carry 7 marks.

Part A — Short Questions (2 Marks)

Q1. (2 marks) Define a tree. Give an example. **Solution — full explanation.**

What the question asks: the definition of a tree and one example. **Definition.** A **tree** is a connected undirected graph that contains **no simple circuit** (no cycle). Equivalently, a tree is a simple graph in which there is a **unique path** between every pair of vertices. **Example.** The graph below (root r with children a, b, c) is a tree: it is connected and has no cycle.



Final answer. A tree is a connected acyclic graph; equivalently, a graph with a unique path between every pair of vertices.

Q2. (2 marks) State any two properties of a tree with n vertices. **Solution — full explanation.**

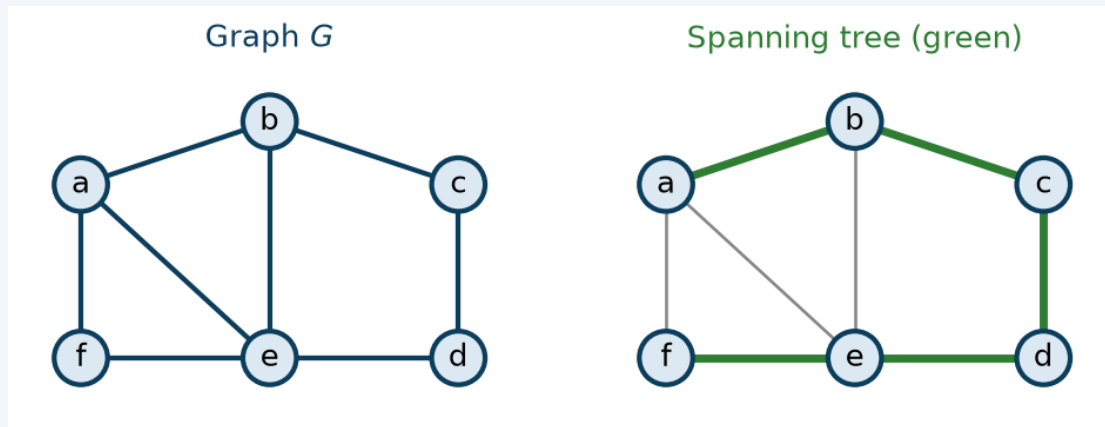
What the question asks: two facts that every tree satisfies. **Properties.**

- A tree with n vertices has exactly **$n - 1$ edges**.
- A tree is connected and acyclic; adding any edge creates exactly one cycle, and removing any edge disconnects it.

Final answer. (i) n vertices $\Rightarrow n - 1$ edges; (ii) adding an edge creates a cycle and removing an edge disconnects the tree.

Q3. (2 marks) Define a spanning tree of a graph. **Solution — full explanation.**

What the question asks: define a spanning tree. **Definition.** Given a connected graph G , a **spanning tree** is a subgraph T of G that contains **all the vertices** of G and is itself a tree. It therefore has n vertices and $n - 1$ edges.

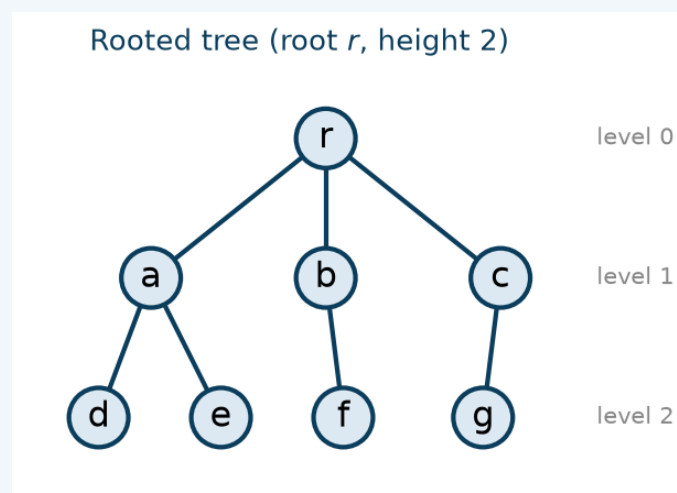


Final answer. A spanning tree is a tree subgraph that keeps every vertex of G (and has $n - 1$ edges).

Q4. (2 marks) Define a rooted tree, the level of a vertex and the height of a tree. **Solution — full explanation.**

What the question asks: three related definitions. **Definitions.**

- **Rooted tree:** a tree in which one vertex is designated the **root**; edges are viewed as directed away from the root.
- **Level $\ell(v)$:** the length of the simple path from v to the root.
- **Height h :** the maximum of all the level numbers, $h = \max_v \ell(v)$.



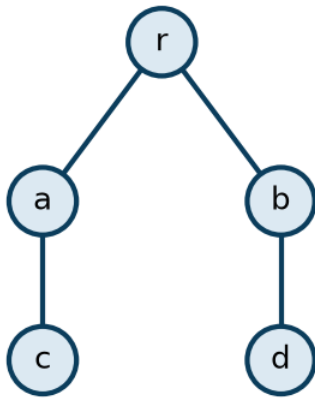
Final answer. Rooted tree = tree with a root; level = distance from the root; height = largest level.

Q5. (2 marks) Define a binary tree and a full binary tree. **Solution — full explanation.**

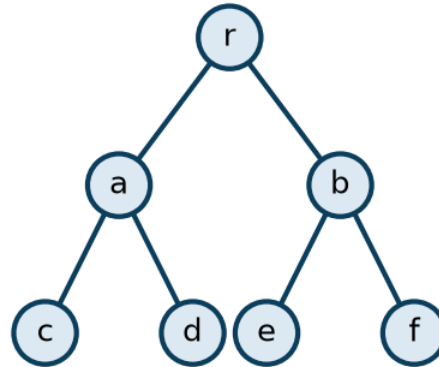
What the question asks: two definitions of rooted trees. **Definitions.**

- **Binary tree:** a rooted tree in which every vertex has **zero, one or two** children.
- **Full binary tree:** a binary tree in which every vertex has **either two children or none**.

Binary tree



Full binary tree ($k=2$ internal, 3 leaves)



Final answer. A binary tree allows 0, 1 or 2 children; a full binary tree allows only 0 or 2.

Q6. (2 marks) If a full binary tree has k internal vertices, how many terminal vertices and total vertices does it have? **Solution — full explanation.**

What the question asks: count the leaves and vertices of a full binary tree. **Theorem.** A full binary tree with k internal vertices has

$$\text{terminal vertices} = k + 1, \quad \text{total vertices} = 2k + 1.$$

Why. Each internal vertex has 2 children, so there are $2k$ edges. A tree with V vertices has $V - 1$ edges, so $V - 1 = 2k$, giving $V = 2k + 1$; the leaves are the non-internal ones, $V - k = k + 1$.

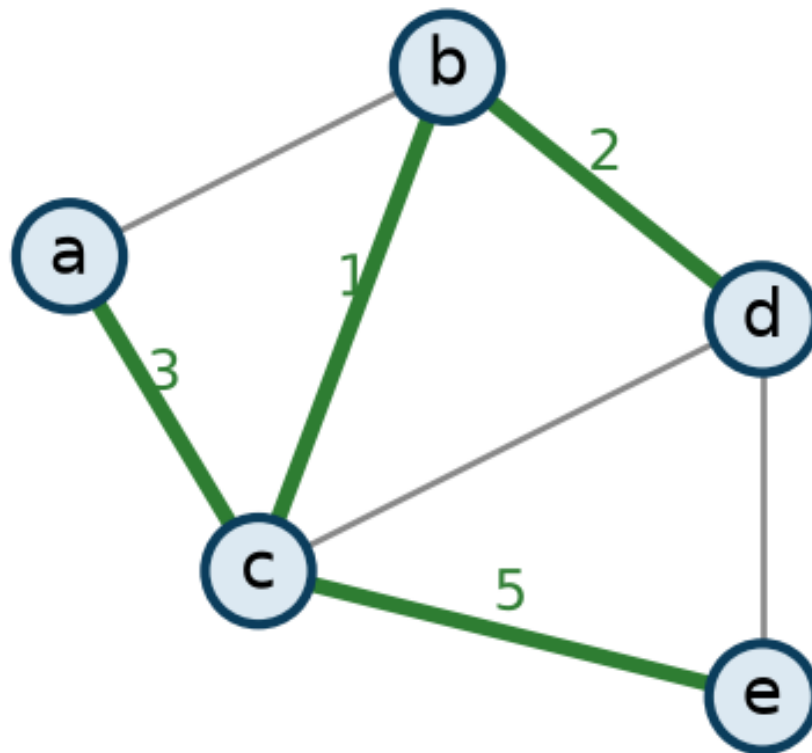
Final answer. $k + 1$ terminal vertices and $2k + 1$ total vertices.

Q7. (2 marks) Define a weighted graph and a minimum spanning tree. **Solution — full explanation.**

What the question asks: two definitions used by Kruskal and Prim. **Definitions.**

- **Weighted graph:** a graph in which each edge e carries a non-negative number $w(e)$ (its weight).
- **Minimum spanning tree (MST):** a spanning tree whose total weight (sum of its edge weights) is as small as possible.

Minimum spanning tree (weight 11)



Final answer. A weighted graph has weights on edges; an MST is the spanning tree of least total weight.

Q8. (2 marks) State the conditions under which Dijkstra's algorithm can be applied. **Solution — full explanation.**

What the question asks: when Dijkstra is valid. **Conditions.**

- The graph must be **connected**.
- All edge weights must be **non-negative** (no negative weights).
- It works for both directed and undirected graphs.

Final answer. Dijkstra requires a connected graph with non-negative edge weights.

Q9. (2 marks) Define a flow network, capacity and flow conservation. **Solution — full explanation.**

What the question asks: the basic definitions of network flow. **Definitions.**

- **Flow network:** a directed graph with a non-negative **capacity** $c(u, v)$ on each edge, a source s and a sink t .
- **Capacity constraint:** the flow must satisfy $0 \leq f(u, v) \leq c(u, v)$.
- **Flow conservation:** at every vertex except s and t , $\sum_v f(u, v) = 0$ (inflow = outflow).

Final answer. A flow network is a capacitated digraph with a source and a sink; flows obey the capacity constraint and conservation.

Q10. (2 marks) What is an augmenting path? Define residual capacity. **Solution — full explanation.**

What the question asks: two ideas behind Ford-Fulkerson. **Definitions.**

- **Residual capacity:** the extra flow that can still be pushed along an edge, $c_f(u, v) = c(u, v) - f(u, v)$.
- **Augmenting path:** a path from s to t in the residual network along which more flow can be pushed; the amount pushed is the path's bottleneck (smallest residual capacity on it).

Final answer. Residual capacity $= c - f$; an augmenting path is a residual s -to- t path that increases the flow.

Q11. (2 marks) State the Max-Flow Min-Cut theorem. **Solution — full explanation.**

What the question asks: state the theorem linking flows and cuts. **Theorem.** In any flow network, the value of a **maximum flow** equals the capacity of a **minimum cut**:

$$\max |f| = \min_{(S,T)} \text{cap}(S, T),$$

where an s - t cut partitions the vertices into S (containing s) and T (containing t), and $\text{cap}(S, T) = \sum_{u \in S, v \in T} c(u, v)$. **Final answer.** Max-flow value = min-cut capacity.

Q12. (2 marks) Differentiate between DFS and BFS. **Solution — full explanation.**

What the question asks: the key difference between the two traversals.

DFS	BFS
Goes deep along one branch before backtracking	Visits level by level
Uses a stack (or recursion)	Uses a queue
Produces a spanning tree that is often a long path	Produces a spanning tree whose levels equal graph distance

Final answer. DFS explores depth-first (stack); BFS explores breadth-first (queue).

Part B — Long Questions (7 Marks)

Q13. (7 marks) Prove that a tree with n vertices has exactly $n - 1$ edges. **Solution — full explanation.**

What the question asks: prove the edge count of a tree. **Proof.** Start with the n vertices and no edges: this is n components. Now add edges one at a time, each time joining two vertices that lie in **different** components.

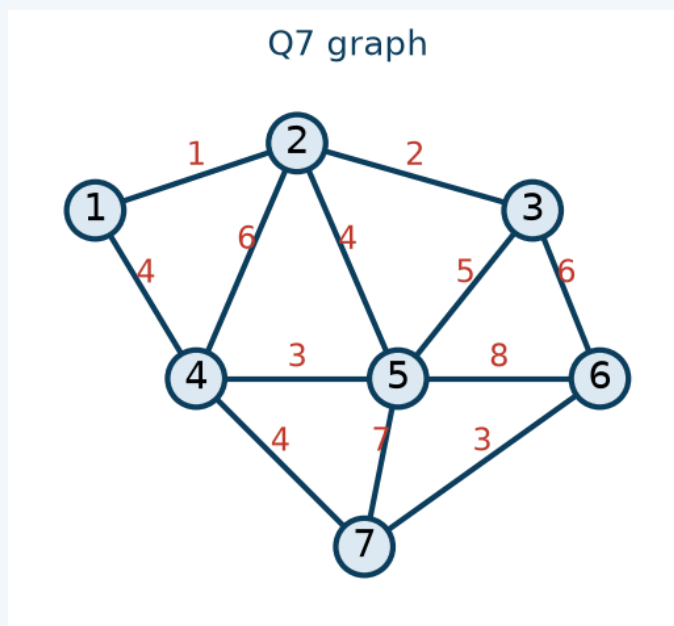
- Each such edge keeps the graph acyclic (it does not create a cycle, because its endpoints are in different components) and **reduces the number of components by 1**.
- To reduce n components down to 1 (i.e. to make the graph connected), we must add exactly $n - 1$ such edges.
- Adding any further edge would have to join two vertices already in the same component, which

would create a cycle — and a tree has no cycles.

Hence a tree with n vertices has exactly $n - 1$ edges. ■ **Final answer.** A tree with n vertices has $n - 1$ edges.

Q14. (7 marks) Use Kruskal's algorithm to find a minimum spanning tree (MST) of the given graph. **Solution — full explanation.**

What the question asks: the least-weight spanning tree. **Kruskal's idea and why.** Sort the edges by weight and repeatedly take the cheapest edge that does **not** form a cycle. Taking the cheapest non-cycling edge at each step is guaranteed to give an MST (greedy choice).



Step-by-step.

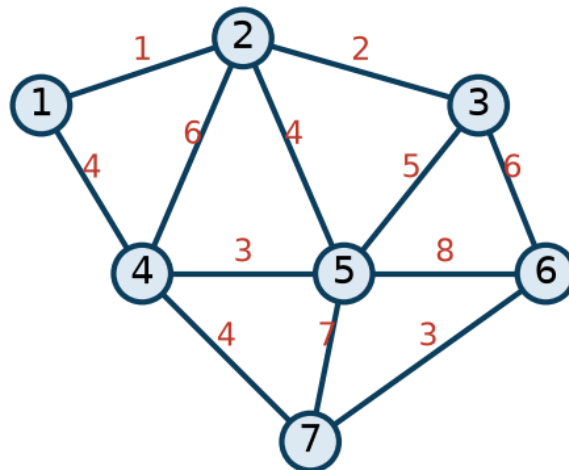
Weight	Edge	Take?
1	1-2	yes
2	2-3	yes
3	4-5	yes
3	6-7	yes
4	1-4	yes
4	2-5	no (forms a cycle)
4	4-7	yes

We now have $n - 1 = 6$ edges: 1-2, 2-3, 4-5, 6-7, 1-4, 4-7. **Final answer.** MST weight = $1 + 2 + 3 + 3 + 4 + 4 = \boxed{17}$.

Q15. (7 marks) Use Prim's algorithm to find a minimum spanning tree of the same graph. **Solution — full explanation.**

What the question asks: the MST by Prim's method. **Prim's idea and why.** Grow a single tree from a starting vertex, each time adding the **cheapest edge** that joins the current tree to a vertex outside it.

Q7 graph



Step-by-step (start at vertex 1).

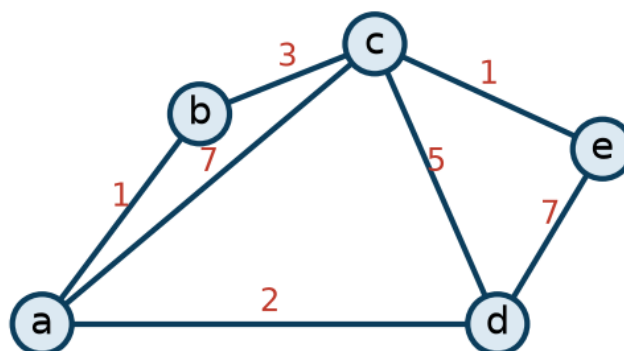
- From 1: cheapest edge is 1–2 (1). Take it. Tree {1,2}.
- From {1,2}: cheapest outgoing is 2–3 (2). Take it. Tree {1,2,3}.
- From {1,2,3}: cheapest outgoing is 1–4 (4). Take it. Tree {1,2,3,4}.
- From {1,2,3,4}: cheapest outgoing is 4–5 (3). Take it. Tree {1,2,3,4,5}.
- From {1,2,3,4,5}: cheapest outgoing is 4–7 (4). Take it. Tree {1,...,7} except 6.
- From the tree: cheapest edge to 6 is 6–7 (3). Take it.

Edges: 1-2, 2-3, 1-4, 4-5, 4-7, 6-7. **Final answer.** MST weight = $1 + 2 + 4 + 3 + 4 + 3 = 17$ (same total as Kruskal — an MST need not be unique, but its weight is).

Q16. (7 marks) Use Dijkstra's algorithm to find the shortest distance and shortest path from a to e . **Solution — full explanation.**

What the question asks: the least-weight route from a to e . **Dijkstra's idea and why.** Repeatedly settle the unsettled vertex with the smallest known distance, then relax its neighbours.

Q2 graph



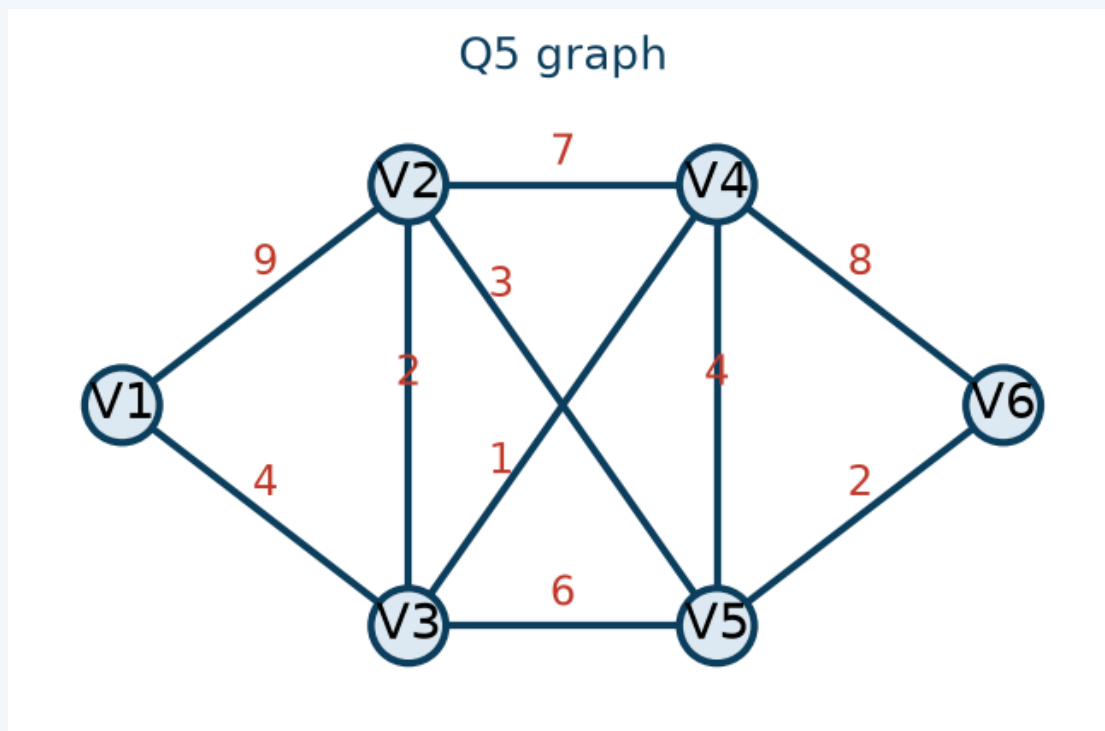
Step-by-step (source a).

- $a = 0$; neighbours: $b = 1$, $c = 7$, $d = 2$.
- Settle b (1): relax c via b : $1 + 3 = 4 < 7 \Rightarrow c = 4$.
- Settle d (2): via d , c would be $2 + 5 = 7 > 4$ (no change); e via d : $2 + 7 = 9$.
- Settle c (4): relax e via c : $4 + 1 = 5 < 9 \Rightarrow e = 5$.
- Settle e (5).

Distances: $b = 1$, $d = 2$, $c = 4$, $e = 5$. **Final answer.** Shortest distance $a \rightarrow e = \boxed{5}$, along the path $a \rightarrow b \rightarrow c \rightarrow e$.

Q17. (7 marks) Use Dijkstra's algorithm to find the shortest path between V_1 and V_6 . **Solution — full explanation.**

What the question asks: the least-weight route from V_1 to V_6 .



Step-by-step (source V_1).

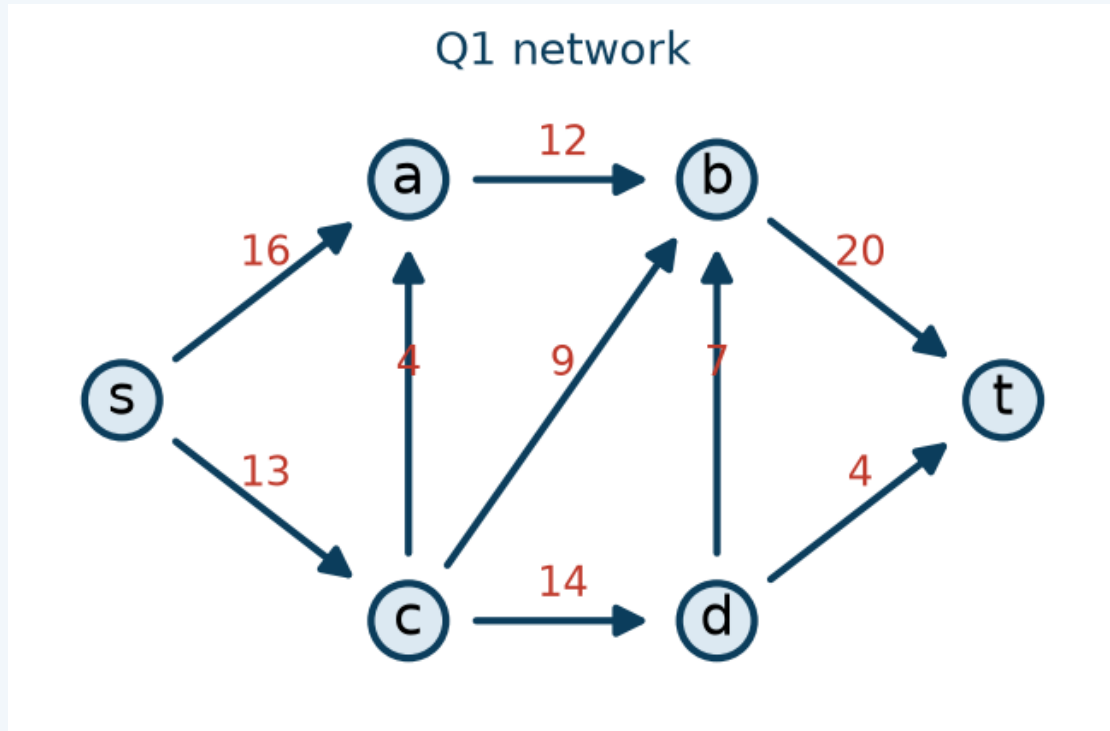
- $V_1 = 0$; neighbours: $V_2 = 9$, $V_3 = 4$.
- Settle V_3 (4): V_2 via V_3 : $4 + 2 = 6 < 9 \Rightarrow V_2 = 6$; $V_4 = 4 + 1 = 5$; $V_5 = 4 + 6 = 10$.
- Settle V_4 (5): V_5 via V_4 : $5 + 4 = 9 < 10 \Rightarrow V_5 = 9$; $V_6 = 5 + 8 = 13$.
- Settle V_2 (6): V_5 via V_2 : $6 + 3 = 9$ (no change).
- Settle V_5 (9): V_6 via V_5 : $9 + 2 = 11 < 13 \Rightarrow V_6 = 11$.
- Settle V_6 (11).

Final answer. Shortest distance $V_1 \rightarrow V_6 = \boxed{11}$, along the path $V_1 \rightarrow V_3 \rightarrow V_4 \rightarrow V_5 \rightarrow V_6$.

Q18. (7 marks) Find the maximum flow in the transport network shown, and determine the corresponding minimum cut. **Solution — full explanation.**

What the question asks: the largest flow from s to t , and a cut of equal capacity. **Method and why.** Use Ford-Fulkerson: push flow along augmenting paths until none remains. By the

Max-Flow Min-Cut theorem, the resulting flow value equals the minimum cut capacity.



Max flow. The two edges into t are $b \rightarrow t$ (capacity 20) and $d \rightarrow t$ (capacity 4). We can saturate both:

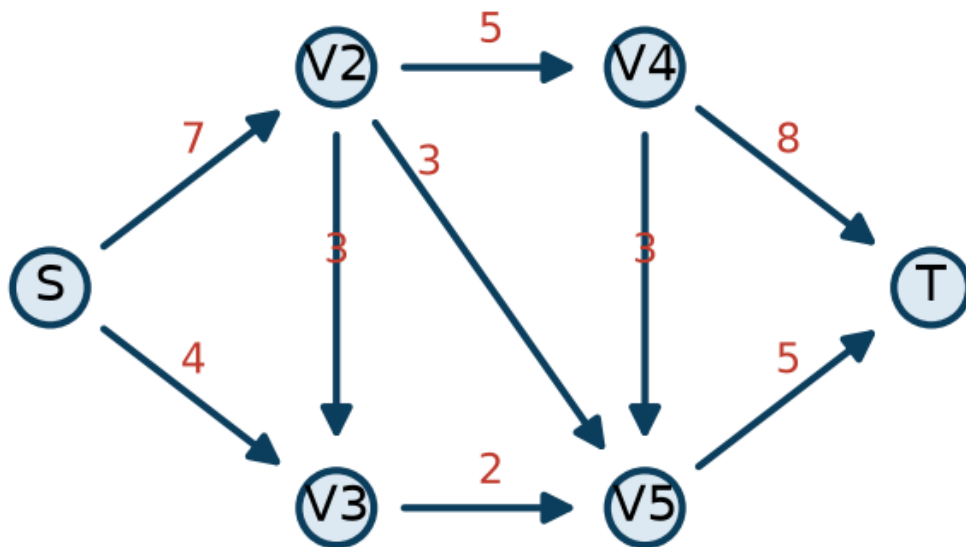
- $s \rightarrow a \rightarrow b \rightarrow t$: push 11 (limited downstream), and $s \rightarrow c \rightarrow b \rightarrow t$: push 2, and $s \rightarrow c \rightarrow d \rightarrow b \rightarrow t$: push 7;
- $s \rightarrow c \rightarrow d \rightarrow t$: push 4.

Total into t : $b \rightarrow t = 20$, $d \rightarrow t = 4$, giving $20 + 4 = 24$. **Minimum cut.** Take $S = \{s, a, b, c, d\}$, $T = \{t\}$. Edges from S to T : $b \rightarrow t$ (20) and $d \rightarrow t$ (4), so $\text{cap}(S, T) = 24$. **Final answer.** Maximum flow = 24; minimum cut = $\{b \rightarrow t, d \rightarrow t\}$ with capacity 24.

Q19. (7 marks) Find the maximum flow in the network using the Ford-Fulkerson algorithm. **Solution — full explanation.**

What the question asks: the largest flow from S to T . **Method.** Push flow along augmenting paths, updating the residual network, until no augmenting path remains.

Q6 network



Augmenting paths.

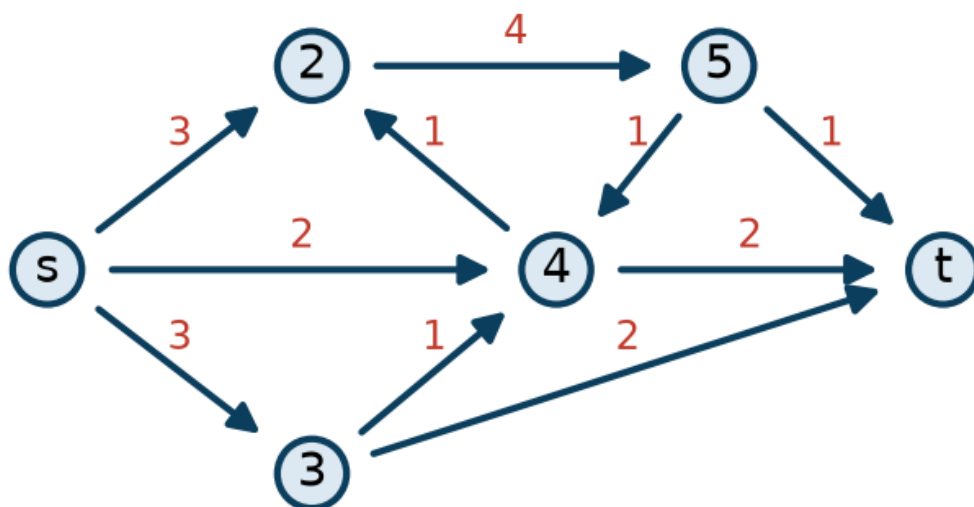
- $S \rightarrow V_2 \rightarrow V_4 \rightarrow T$: bottleneck 5 $\Rightarrow$ push 5.
- $S \rightarrow V_2 \rightarrow V_5 \rightarrow T$: bottleneck 2 $\Rightarrow$ push 2.
- $S \rightarrow V_3 \rightarrow V_5 \rightarrow T$: bottleneck 2 $\Rightarrow$ push 2.

Total into T : $V_4 \rightarrow T = 4$ and $V_5 \rightarrow T = 5$, giving $4 + 5 = 9$. No augmenting path remains, so this is maximum. **Final answer.** Maximum flow = 9.

Q20. (7 marks) Find the maximal flow of the network using the Ford-Fulkerson algorithm. **Solution — full explanation.**

What the question asks: the largest flow from s to t .

Q8 network



Upper bound and why. The only edges entering t are $5 \rightarrow t$ (1), $4 \rightarrow t$ (2) and $3 \rightarrow t$ (2), so no flow can exceed $1 + 2 + 2 = 5$. **Achieve 5.**

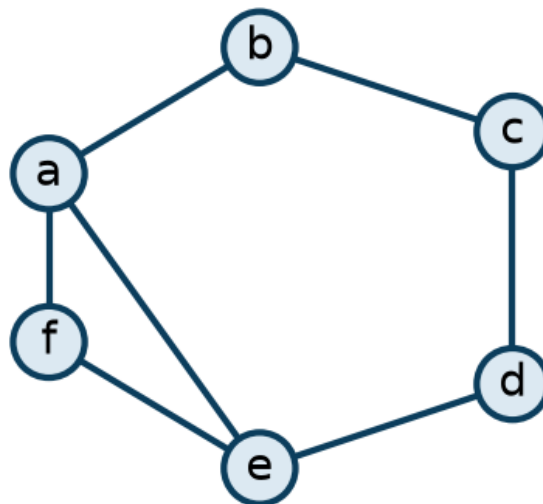
- $s \rightarrow 2 \rightarrow 5 \rightarrow t$: push 1 (limited by $5 \rightarrow t$).
- $s \rightarrow 4 \rightarrow t$: push 2.
- $s \rightarrow 3 \rightarrow t$: push 2.

Total = 5, matching the cut $\{s, 2, 3, 4, 5\} \rightarrow \{t\}$ whose capacity is $1 + 2 + 2 = 5$. **Final answer.** Maximum flow = $\boxed{5}$.

Q21. (7 marks) Explain depth-first search (DFS) and breadth-first search (BFS), and show the spanning tree each produces for the graph below. **Solution — full explanation.**

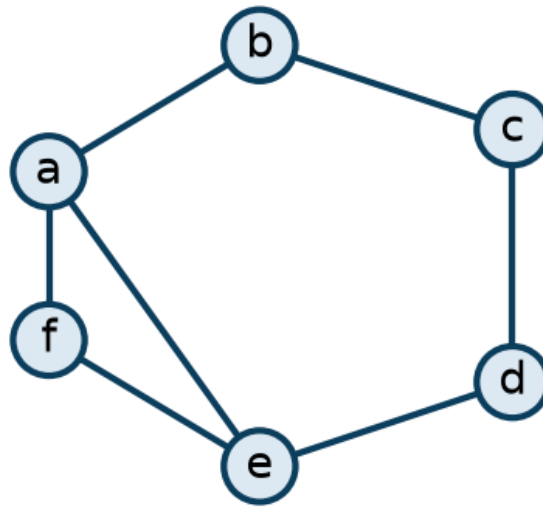
What the question asks: describe both traversals and apply them. **DFS.** Go **deep** along one branch before backtracking; use a **stack** (or recursion). At each step move to an unvisited neighbour; when stuck, backtrack.

DFS from a: a, b, c, d, e, f



BFS. Visit **level by level** using a **queue**: visit all neighbours of the current vertex, then their neighbours, and so on.

BFS from a : a, b, f, e, c, d



Spanning trees. In both cases, the edges used to reach a *new* vertex form a spanning tree of the graph. For this graph:

- DFS from a visits a, b, c, d, e, f (a long path), giving a spanning tree that is a path.
- BFS from a visits a, b, f, e, c, d (level by level), giving a shallow spanning tree rooted at a .

Final answer. DFS goes deep (stack); BFS goes wide (queue). Both yield a spanning tree; BFS also gives the unweighted shortest-path tree from the root.